



使用JDBC连接数据库

北京理工大学计算机学院
金旭亮

Java数据库应用程序全局视图



JDBC规定了一整套访问数据库的标准API，所有数据库都需要实现它，因此，使用JDBC访问数据库的Java应用程序，是很容易切换底层数据库的。

JDBC核心类型一览表



核心类型 (java.sql)	说明
DriverManager	负责装载/卸载驱动程序
Connection	与数据库建立连接
Statement	在一个给定的连接中执行SQL语句
PreparedStatement	用于执行预编译的SQL命令
CallableStatement	用于调用数据库中存储过程
ResultSet	保存SQL命令的执行结果

上述组件是独立于底层数据库的，也就是说，只要连接上了数据库，相同的代码，就可以顺利工作.....

JDBC访问数据库的基本步骤



加载JDBC驱动程序

创建数据库连接

执行SQL语句

接收并处理SQL的返回结果

关闭创建的各个对象



对于有可视化界面的应用程序，或者是Server端应用程序，应该在独立的线程中完成这些步骤。

出于精简学习负担的目的，我们将以SQLite为例介绍JDBC的基本使用，在此基础上，后面选择微软的SQL Server来介绍JDBC的高级特性……



JDBC连接SQLite数据库

下载SQLite的JDBC驱动-1

SQLite官网上没有直接提供java的驱动程序组件包，可以通过搜索引擎搜索“sqlite jdbc”找到相应的包。



SQLite JDBC驱动源码
托管于GitHub

也可以直接访问Maven
组件库下载jar包

下载SQLite的JDBC驱动-2

也可以访问以下网址，直接下载SQLite的JDBC驱动：

ReleasesTags

Latest release

3.36.0.3

5bc86e1

Verified

Compare

https://github.com/xerial/sqlite-jdbc/releases

3.36.0.3

xerial released this 17 days ago

- Fixes for GraalVM
- Internal update: Migrate to JUnit5. Add CI with GraalVM

Assets 3

sqlite-jdbc-3.36.0.3.jar

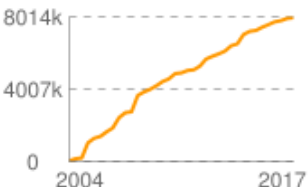
Source code (zip)

Source code (tar.gz)

下载SQLite的JDBC驱动-3

<http://mvnrepository.com/artifact/org.xerial/sqlite-jdbc>

MVNREPOSITORY
Search for groups, artifacts, categories

Indexed Artifacts (8.02M)


Popular Categories

- Aspect Oriented
- Actor Frameworks
- Application Metrics
- Build Tools
- Bytecode Libraries
- Command Line Parsers
- Cache Implementations
- Cloud Computing
- Code Analyzers
- Collections
- Configuration Libraries
- Core Utilities
- Date and Time Utilities

Home » org.xerial » sqlite-jdbc

**SQLite JDBC**
SQLite JDBC library

License	Apache 2.0
Tags	sqlite jdbc sql
Used By	390 artifacts

**QNB**
qnb.com

点这里进入相应版本的下载页面

Home » org.xerial » sqlite-jdbc » 3.20.1

**SQLite JDBC » 3.20.1**
SQLite JDBC library

License	Apache 2.0
Date	(Oct 08, 2017)
Files	Download (JAR) (6.3 MB)
Repositories	Central
Used By	390 artifacts

[Maven](#) [Gradle](#) [SBT](#) [Ivy](#) [Grape](#) [Leiningen](#) [Buildr](#)

```
// https://mvnrepository.com/artifact/org.xerial/sqlite-jdbc
testCompile group: 'org.xerial', name: 'sqlite-jdbc', version: '3.20.1'
```

也可以通过Maven、Gradle联机下载

☒ Include comment with link to declaration

Version	Repository	Usages	Date	
3.20.x	3.20.1	Central	2	(Oct, 2017)
	3.20.0	Central	15	(Aug, 2017)
3.19.x	3.19.3	Central	36	(Jun, 2017)
3.18.x	3.18.0	Central	15	(May, 2017)
3.16.x	3.16.1	Central	56	(Jan, 2017)

GitHub repository page for **xerial / sqlite-jdbc**. The page shows the repository name, navigation tabs (Code, Issues, Pull requests, Projects, Insights), and a "Join GitHub today" banner.

SQLite JDBC Driver

sqlite jdbc java database

980 commits 26 branches 26 releases 45 contributors Apache-2.0

Branch: master New pull request Find file Clone or download

xerial Next snapshot version

.settings	Modified documentation to create more consistency, remove
archive	added regex3.8a
demo	mac native library
docker	Add support for aarch64 (arm64)

Clone with HTTPS ?

Use Git or checkout with SVN using the web URL.

<https://github.com/xerial/sqlite-jdbc.git>

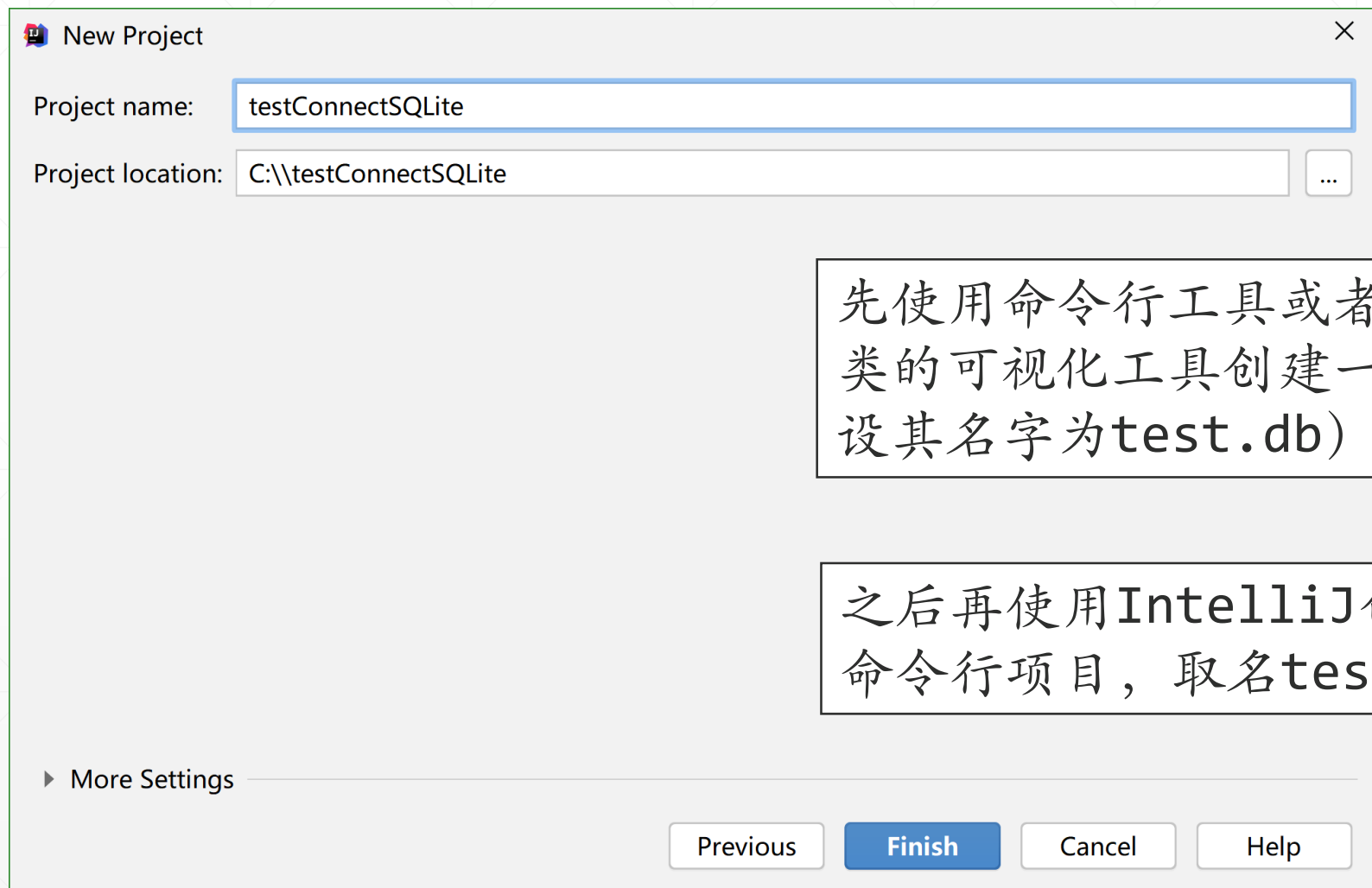
Open in Desktop Download ZIP

托管于GitHub上的SQLite JDBC源码

<https://github.com/xerial/sqlite-jdbc>

点这里下载源码zip包以供学习和研究

编写代码使用JDBC访问SQLite

A screenshot of the 'New Project' dialog box in IntelliJ IDEA. The dialog has a title bar with a small icon and the text 'New Project' and a close button. It contains two input fields: 'Project name:' with the text 'testConnectSQLite' and 'Project location:' with the text 'C:\\testConnectSQLite'. Below these fields is a 'More Settings' link. At the bottom are four buttons: 'Previous', 'Finish' (highlighted in blue), 'Cancel', and 'Help'.

New Project

Project name: testConnectSQLite

Project location: C:\\testConnectSQLite

► More Settings

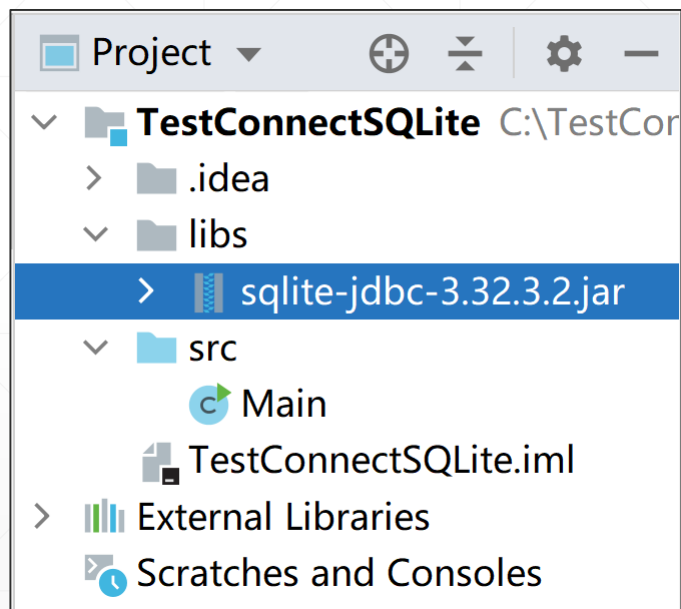
Previous Finish Cancel Help

先使用命令行工具或者诸如DB Browser之类的可视化工具创建一个测试数据库（假设其名字为test.db）

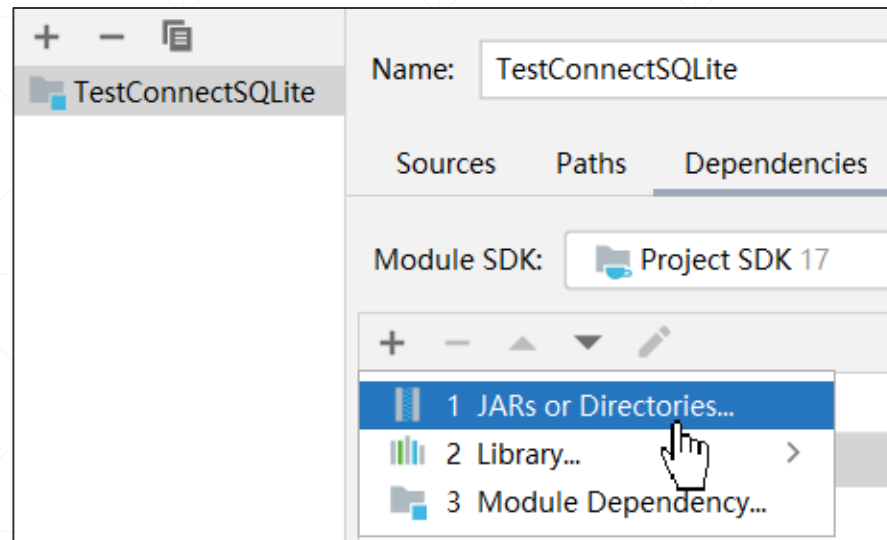
之后再使用IntelliJ创建一个普通的Java命令行项目，取名testConnectSQLite

添加JDBC驱动到项目中

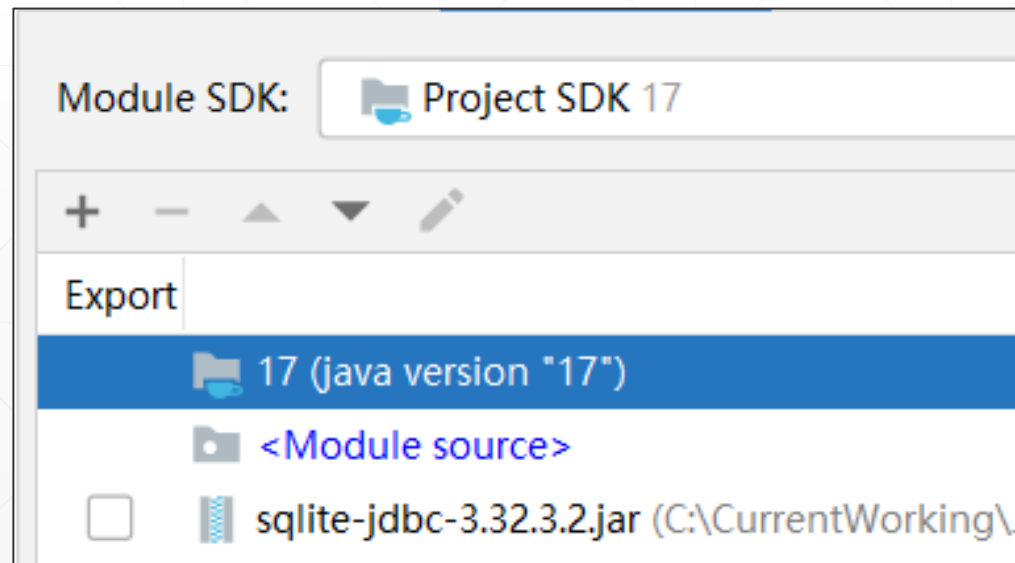
给项目创建一个libs文件夹，从网上下载SQLite的JDBC组件包，复制到此文件夹中：



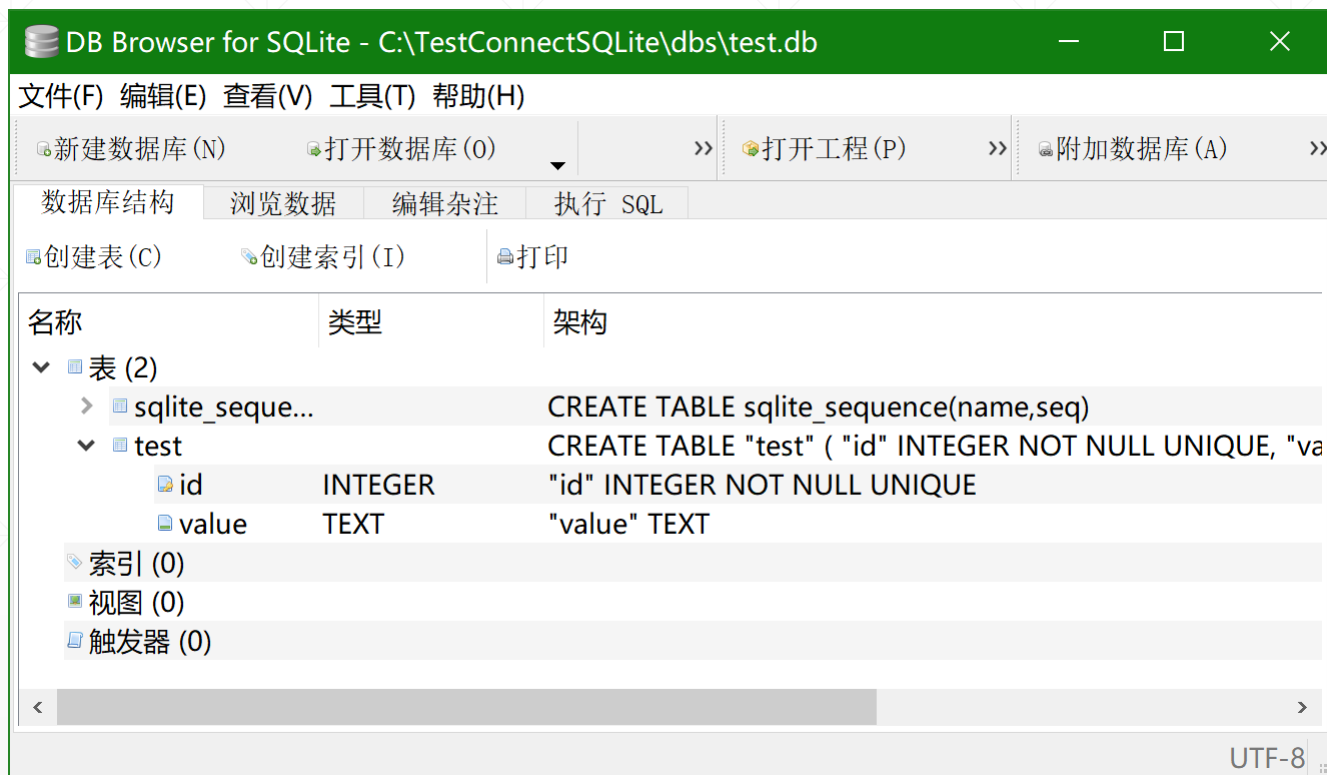
打开项目的Project Structure对话框设置当前模块依赖于此jar包。



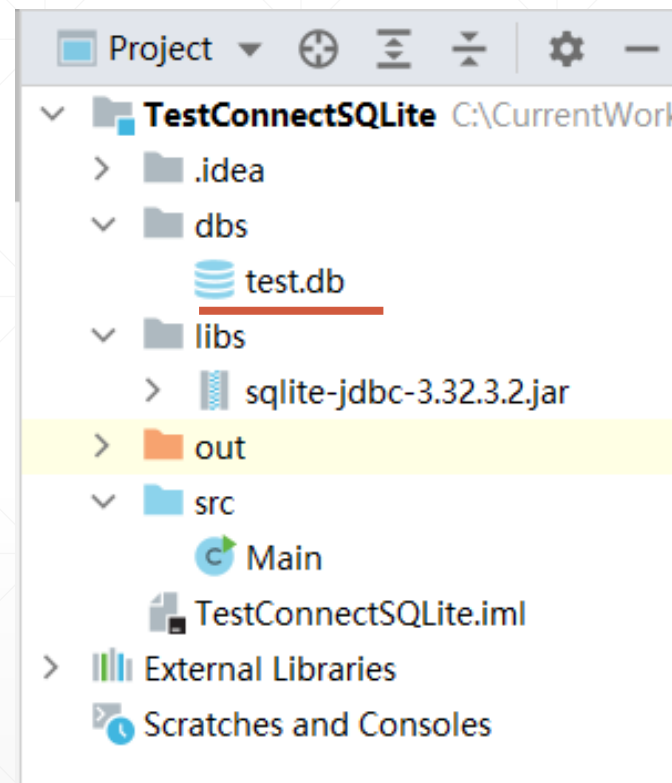
给模块添加对jar包的引用



准备数据库



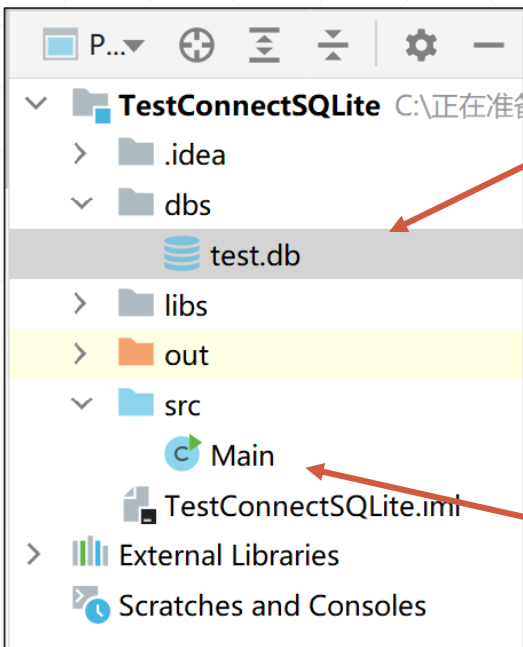
使用DB Browser创建一个示例数据库test.db，在里面创建一个test表，定义两个字段，然后，随意添加几条记录作为测试用例。



示例数据库名为test.db，放在项目的dbs文件夹下

将数据库加入到项目中

创建dbs文件夹，将test.db复制到它下面：



注意一下访问test.db的路径是：
./dbs/test.db

路径是以IntelliJ项目所在的文件夹为起点确定的。

创建一个包容程序入口点的Main类，
准备编写数据库连接代码。

连接数据库



`DriverManager`类代表驱动程序管理器，提供了几个静态方法负责装载驱动程序和得到对特定数据库的连接，其中`getConnection`方法用于连接数据库。

```
Connection connection = DriverManager.getConnection("jdbc:sqlite:./dbs/test.db");
```

数据库连接URL

JDBC URL



JDBC URL具有特定的结构，封装了相应的信息，可用于加载相应的驱动程序并与数据库建立连接。标准的JDBC URL的格式如下：

jdbc : <子协议名> : <子名称>

JDBC URL由三个部分组成，各个部分之间用冒号分隔。<子协议>是指数据库连接的方式。<子名称>可以根据子协议的改变而变化。

示例：连接本机MySQL数据库testDB的JDBC URL

jdbc:mysql://localhost/testDB

测试用Java代码连接数据库并提取数据

```
private static void testConnection() {  
    Connection connection = null;  
    try {  
        // 创建数据库连接  
        connection = DriverManager.getConnection(  
            url: "jdbc:sqlite:./dbs/test.db ");  
        // 准备创建操作与存取的数据库命令  
        Statement statement = connection.createStatement();  
        // 查询表中保存的数据  
        ResultSet rs = statement.executeQuery( sql: "select * from test");  
        while (rs.next()) {  
            // 输出查询到的数据  
            System.out.println("-----");  
            System.out.println("id = " + rs.getInt( columnLabel: "id"));  
            System.out.println("value = " + rs.getString( columnLabel: "value"));  
        }  
    }  
    catch (SQLException e) {...}  
    finally {...}  
}
```

注意一下数据库连接
字符串



Run: Main x

```
"C:\Program Files\Java\jdk-15\bin\java.exe"  
-----  
id = 1  
value = One  
-----  
id = 2  
value = Two  
-----  
id = 3  
value = Three  
  
Process finished with exit code 0
```

太棒了，你成功地完成了第一个Java数据库应用程序！

麻烦的关闭代码

```
private static void testConnection() {  
    Connection connection = null;  
    try {  
        // 创建数据库连接  
        connection = DriverManager.getConnection(  
            url: "jdbc:sqlite:./dbs/test.db ");  
        // 准备创建操作与存取的数据库命令  
        Statement statement = connection.createStatement();  
        // 查询表中保存的数据  
        ResultSet rs = statement.executeQuery( sql: "select * from test");  
        while (rs.next()) {...}  
    }  
    catch (SQLException e) {  
        System.err.println(e.getMessage());  
    }  
    finally {  
        try {  
            if (connection != null) connection.close();  
        } catch (SQLException e) {  
            System.err.println(e);  
        }  
    }  
}
```

事实上，不仅Connection需要关闭，Statement和ResultSet其实也是需要即时关闭的.....

由于数据库连接是一种比较昂贵的资源，所以，必须保证关闭。

由于Connection对象的close()方法会抛出异常，所以不得不嵌套try/catch结构.....

简化代码.....

```
private static void testConnectionSimple() {  
    String dbUrl = "jdbc:sqlite:./dbs/test.db";  
    //使用try-with-resources简化JDBC连接数据库代码  
    try (Connection connection = DriverManager.getConnection(dbUrl);  
         Statement statement = connection.createStatement();  
         ResultSet rs = statement.executeQuery(sql: "select * from test");  
    ) {  
        while (rs.next()) {  
            // 输出查询到的数据  
            System.out.println("-----");  
            System.out.println("id = " + rs.getInt(columnLabel: "id"));  
            System.out.println("value = " + rs.getString(columnLabel: "value"));  
        }  
    } catch (SQLException e) {  
        System.err.println("数据库连接失败: " + e.getMessage());  
    }  
}
```

将需要自动关闭的对象放到try表达式中，这是推荐的编码方式。

当程序执行流程离开try语句块时，会自动调用它所监控对象的close()方法。

多知道一点：早期的数据库驱动加载代码



早期的JDBC代码，使用以下这句加载特定数据库的数据驱动：

```
Class.forName("com.mysql.jdbc.Driver");
```

JDBC 4.0以后版本的DriverManager，它的getConnection方法进行了增强，只需要传给它数据库连接字符串，它就可以自动进行推断并加载相应的数据库驱动程序。